CAN-Sentinel

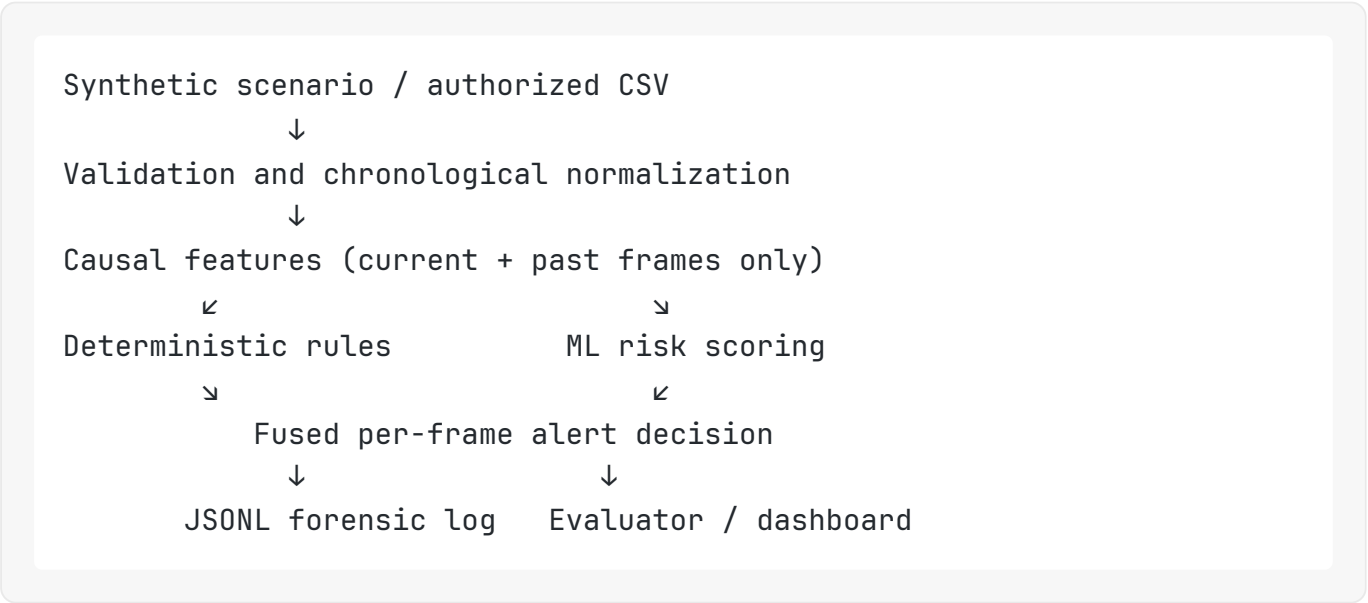
Technical Guide: How the Automotive ECU Intrusion-Detection Demo Works

- **Audience:** Security Admin Associate L1 / Cyber Security internship
- **Scope:** Offline CAN 2.0B analysis and synthetic attack simulation

> **Safety statement:** This educational prototype does not open a CAN adapter, communicate with a vehicle, or transmit frames. It is not production-ready, safety-certified, or hard-real-time software. Synthetic metrics must not be represented as vehicle performance.

1. At a glance

CAN-Sentinel demonstrates an offline intrusion-detection workflow. It generates benign and attack-like records or parses a CSV capture, validates the fields, derives causal per-frame features, applies deterministic rules and a machine-learning risk score, records alerts, and displays results in Streamlit.



2. Automotive CAN security background

CAN is a message-oriented in-vehicle network used by electronic control units (ECUs). A Classical CAN frame carries an arbitration identifier, a data-length code (DLC), and up to eight

data bytes. Lower numeric identifiers win arbitration when senders contend for the bus, so high-priority traffic can delay lower-priority frames.

A CAN identifier is not a cryptographic proof of sender identity. Depending on network access, a compromised node may send frames using another ECU's identifier. Monitoring can therefore look for unexpected IDs, rates, timing, or payload patterns. CAN-Sentinel illustrates such checks only. It has no OEM signal database, cryptographic sender authentication, vehicle gateway, safety response, or validated vehicle baseline.

3. Repository map

```
CAN-Sentinel/
├── app.py                          Streamlit simulation / CSV dashboard
├── generate_report.py              Project summary and vulnerability repor
├── README.md                       Quick start and user workflow
├── CAN-Sentinel-Project-Report.md Concise vulnerability assessment
├── CAN-Sentinel-Technical-Guide.md This walkthrough
├── CAN-Sentinel-Technical-Guide.pdf PDF version of this guide
├── requirements.txt                Runtime dependencies
├── requirements-dev.txt            Runtime dependencies plus pytest
├── can_sentinel/
│   ├── simulator.py                 Typed frame and synthetic scenario ger
│   ├── preprocessor.py              CSV normalization and causal features
│   ├── detector_rules.py             Rolling-window protocol/rate rules
│   ├── detector_ml.py                Isolation Forest / Random Forest scor
│   ├── seasonal_theme.py             Calendar-based display palette
│   ├── engine.py                     Per-frame fusion, timing, JSONL alert
│   └── evaluator.py                  Fused classification + ML ROC-AUC met
└── tests/test_pipeline.py            Regression tests
```

4. Frame format and simulator

`CANFrame` is an immutable record with `timestamp`, `arbitration_id`, `dlc`, `data`, `label`, and `attack`. Construction rejects non-finite/negative timestamps, invalid identifiers, DLC outside integer range 0–8, payload/DLC mismatch, and labels other than 0 (benign) or 1 (attack).

`TrafficSimulator(seed=7)` uses deterministic pseudorandom generation. Reusing a seed and the same calls reproduces the traffic. The generated data is a simplified teaching sample, not a physical bus timing model.

- **Normal:** repeats IDs `0x100`, `0x120`, `0x200`, `0x316`, and `0x400`, with eight-byte payloads. For `0x316`, the first two bytes contain little-endian example RPM values.
- **DoS:** emits identifier `0x000` with default interval one tenth the normal period (10x generator frame rate). The default rules also flag `0x000` as reserved.
- **Fuzzy:** random 11-bit identifiers `0x000` – `0x7FF` with random eight-byte payloads.
- **RPM spoof:** uses ID `0x316` with example RPM payloads in the 7000–9999 range.

`scenario()` combines normal and attack frames, labels them, and sorts chronologically. It does not send frames anywhere.

5. CSV parsing and feature extraction

`read_can_csv(path)` requires `Timestamp`, `Arbitration_ID`, `DLC`, and `Data_Bytes`. IDs may be decimal or `0x`-prefixed. Payload hex may contain spaces, colons, commas, underscores, or hyphens. The parser rejects non-finite/negative timestamps, malformed identifiers, fractional or out-of-range DLC, data-length mismatch, invalid labels, and empty logs. If a CSV contains optional `Label`, each value must be 0 or 1. Out-of-order offline rows are stably sorted by timestamp.

`FeatureExtractor` is stateful and causal: process rows in non-decreasing time order and use a fresh instance per independent trace. It computes:

Feature	Definition
`iat`	Seconds since prior frame; first frame is 0
`id_frequency`	Count of current ID in recent bounded frame window, including this frame
`payload_entropy`	Shannon entropy of byte values; zero for empty data
`dlc`	Data length code
`identifier`	Numeric arbitration ID
`id_novelty`	1 when ID has not appeared earlier in this extractor session, else 0
`rpm_raw`	Little-endian first two bytes for ID `0x316`; otherwise zero

These illustrative features do not use signal engineering, vehicle-specific periods, physical plausibility limits, or a validated ECU specification.

6. Deterministic rules

`RuleDetector` keeps one-second rolling timestamp windows by default and checks:

- **DLC bounds:** DLC outside 0–8 (direct rule use; normal frame objects reject invalid DLC earlier).
- **Reserved ID:** any frame with ID `0x000` by default.
- **Bus flood:** more than the configured aggregate frame rate in the window (default 200 frames/second).
- **Per-ID rate:** more than the configured rate for one identifier (default 80 frames/second).

The stream contract is non-decreasing timestamps; the detector rejects out-of-order frames rather than corrupting its windows. Expired per-ID queues are removed. Defaults are demonstration values, not a standard vehicle allowlist: a legitimate vehicle could trigger them. Production settings must come from authorized specifications and validated benign captures.

7. Machine-learning score

`MLDetector.fit()` creates one chronologically mixed synthetic training run with normal and injected attack frames, then extracts all features through one stateful extractor. This avoids resetting feature history at each class boundary. The training attack families are DoS, fuzzy, and spoof.

Two models are used:

1. **Isolation Forest** is fit on generated benign features and contributes an anomaly-oriented score.
2. **Random Forest** learns benign versus generated attack examples.

A monotonic squashing of the Isolation Forest decision value is blended with Random Forest class confidence to produce a bounded score from 0 to 1. This is a **heuristic ranking/risk score, not a calibrated probability**. The model has seen the same synthetic attack families used in demonstrations; the score is not a zero-day guarantee. Thresholds are not validated operating points. Real evaluation requires independently captured, authorized data, held-out vehicles/sessions/attack families, calibration, drift assessment, and analyst review.

8. Per-frame engine and forensic alerts

Create one `IDSEngine` for one chronological stream. The engine:

1. parses and validates typed frame objects or mapping records;
2. rejects invalid or out-of-order frame timestamps;
3. evaluates deterministic rules;
4. computes causal features and optional ML score;
5. determines an alert if **any rule fires OR** the ML score is at/above threshold;
6. appends each alert as a JSON Lines record.

Each event includes a 1-based `frame_index` unique within the current trace, UTC event time, frame timestamp, identifier, severity, `source` (`rule` or `ml`), rule name, description, risk score, and `detection_latency_ms`. The index—not timestamp uniqueness—is used to count alerted frames when multiple CAN frames share a timestamp. That duration measures rule/feature/model decision work before alert construction and disk I/O. It is not end-to-end latency. Python and general-purpose operating systems are not hard-real-time; observed duration varies with hardware, load, and dependencies.

JSONL is line-oriented and easy to inspect. This sample does not add signed logs, tamper evidence, log rotation, retention controls, trusted timestamps, or access-control management. The dashboard makes alerts downloadable, but does not retain its session's alert log after the session ends.

9. Metrics: what is and is not measured

`evaluate(frames, fresh_engine)` requires a label on every frame and an unused engine. It processes the trace once.

Fused classification metrics use the actual per-frame IDS decision (any rule alert or ML score meeting the configured threshold):

- Accuracy = `(TP + TN) / total frames`.
- Precision = `TP / (TP + FP)`.
- Recall = `TP / (TP + FN)`.
- F1 = harmonic mean of precision and recall.
- False-positive rate = `FP / (FP + TN)`.
- Confusion matrix contains TN, FP, FN, TP.

ROC-AUC is calculated from the ML score alone, not the fused rule/ML decision. It is unavailable without an ML model or when the evaluation has only one class. **Mean/p95 latency** report host-dependent detector decision timing before alert logging.

Accuracy can hide poor minority-class performance. A production assessment should additionally report event-level detection delay, per-attack results, false alarms per operating hour, resource use, and response safety. Results from generated traffic are demonstration-only.

10. Dashboard walkthrough

Install and launch from the repository root:

```
python -m venv .venv
source .venv/bin/activate                # Windows PowerShell: .venv\\Scrip
pip install -r requirements.txt
streamlit run app.py
```

Choose **Synthetic scenario** to set an attack type, benign/attack frame count, and seed, or select **Upload CSV log** to inspect an authorized file. Click **Analyze traffic**. The first use of a seed trains the model; Streamlit caches it for that app session. The packet inspector renders bytes as hex. The alert table marks rule versus ML source. With labels, classification KPIs use the fused IDS decision and the ROC curve shows the ML score only. Without labels, the UI intentionally omits accuracy and ROC. Download the alert feed as JSONL if you need a local artifact. The subtle background palette is selected by calendar month (Northern Hemisphere temperate seasons) and refreshes when the app reruns; it does not change IDS behavior.

This is bounded batch analysis, not a live CAN subscription. Dashboard alert records are not persisted on the server after the app session.

11. Tests and reports

```
pip install -r requirements-dev.txt
python -m pytest -q
python -m compileall -q can_sentinel app.py generate_report.py
python generate_report.py
```

`generate_report.py` writes a concise vulnerability assessment Markdown document. `python generate_report.py --pdf` additionally uses the external `manus-md-to-pdf` converter. The checked-in Technical Guide PDF is this fuller walkthrough. Tests verify basic

reproducibility, validation, chronological feature state, parser behavior, fused metrics, alert logging, labels, and ML score determinism; they do not prove security efficacy or safety.

12. Vulnerability assessment and practical next steps

Concern	Potential impact	Safer next step
No sender authentication on classical CAN	A node may impersonate an ECU identifier	Assess secure gateways/CANsec where supported; protect trust boundaries and keys
Arbitration/availability abuse	Critical timing can be impaired	Validate ECU deadlines and safe failure behavior; review gateway rate controls
Fixed demo thresholds	Legitimate vehicle traffic may be falsely flagged or attacks missed	Develop allowlists and periods from approved OEM requirements and captures
Synthetic-only model	Scores may not transfer to actual vehicles or unseen attacks	Collect labeled, authorized data and evaluate on independent groups
Local JSONL not integrity-protected	Alerts may be altered, lost, or misattributed	Secure collection, storage, clocks, access, retention, and signed exports
No response automation	Detection does not isolate a node or remediate an attack	Keep human/safety governance; hazard-analyze any future response
Host-side latency variation	No timing guarantee	Profile on target, bound queues/work, and review implementation for intended deployment

13. Safe-use statement

Use synthetic examples or vehicle data you are authorized to inspect. Do not connect this demo to public-road vehicles or transmit crafted CAN frames. Any future physical-bus interface would require explicit authorization, isolated hardware-in-the-loop testing, safety approval, and a

separate security/hazard review. This code is an educational prototype—not a certified IDS or a production automotive component.